

08 — Security & Compliance

1. Identity & access

Layer	Control
Password storage	Argon2id (<code>argon2</code> package) — memory-hard, salted per user
Login	Email + password; generic error message on bad credentials (no user-enumeration leak)
JWT access token	HS256, ≤ 15 min TTL, payload <code>{ sub, role, email }</code>
Refresh token	Opaque random string, stored hashed in <code>refresh_tokens</code> , rotated on every use, reuse-detection revokes the chain
Logout	Revokes the supplied refresh token (<code>revokedAt</code>)
Email verification	Single-use token in <code>email_verification_tokens</code>
Password reset	Single-use token in <code>password_reset_tokens</code> with short TTL
RBAC	<code>UserRole</code> enum (<code>CUSTOMER</code> , <code>ADMIN</code> , <code>SUPER_ADMIN</code>), enforced by <code>RolesGuard</code> + <code>@Roles(...)</code> decorators
Customer-data ownership	Every resource service checks <code>userId === req.user.sub</code> before mutation

2. Transport

- HTTPS-only (Azure Container Apps & SWA terminate TLS).
- Helmet middleware sets standard hardening headers (CSP-light, HSTS, X-Content-Type-Options, Referrer-Policy, etc.).
- CORS allowlist driven by `CORS_ORIGINS` env (SWA hostnames only in prod).

3. Input validation

- Global `ValidationPipe({ whitelist: true, forbidNonWhitelisted: true, transform: true })`.
- DTOs use `class-validator` decorators (`@IsEmail`, `@IsInt`, `@Min`, `@MaxLength`, `@Matches`, ...).
- Boot-time env validation via Joi (`@nestjs/config`).
- Prisma parameterised queries everywhere — **no string-concat SQL**.

4. Throttling / abuse

- Global `ThrottlerGuard` with default 60 req/min/IP.
- `/api/auth/*` overridden to **5 req / 15 min / IP** to defeat credential stuffing & brute force.
- Webhook endpoints exempt where signature provides authenticity.

5. Webhook integrity

- Stripe webhook payloads are verified with `STRIPE_WEBHOOK_SECRET`.
- `stripe_events.id` enforces idempotency: replay attempts no-op.
- Tabby/Tamara webhooks follow the same pattern with provider-specific HMAC verification.

6. Secrets management

- All secrets in environment variables; never committed.
- Backend `.env` lists: `DATABASE_URL`, `JWT_SECRET`, `JWT_REFRESH_SECRET`, `STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`, `TABBY_*`, `TAMARA_*`, `SMTP_*`, `BLOB_*`, `APPINSIGHTS_CONNECTION_STRING`, `CORS_ORIGINS`, `ADMIN_BOOTSTRAP_PASSWORD`.
- In Azure: Container Apps `secrets` block (referenced in [infra/main.bicep](#)). Suggested upgrade: source from Azure Key Vault via managed identity for rotation auditability.

7. Data protection

- PII fields (email, phone, addresses) live in PostgreSQL; access requires the user to be authenticated and the row to belong to them, or admin role.
- Soft-deleted users have their PII retained for legal retention periods; a future hard-delete job can purge after the window.
- Backups: Azure-managed daily snapshots; restore drill should be exercised quarterly.
- VAT records (orders + invoices) are immutable post-issuance — only status fields update; financial fields are snapshot.

8. OWASP Top 10 (2021) checklist

#	Risk	Mitigation in this codebase
A01	Broken access control	RBAC + per-resource ownership checks; admin routes role-gated
A02	Cryptographic failures	Argon2id passwords; JWT secrets $\geq$ 32 bytes; TLS only
A03	Injection	Prisma parameterised queries; class-validator on inputs
A04	Insecure design	Snapshot fields for VAT/financials; idempotent webhooks; reservation-based stock

#	Risk	Mitigation in this codebase
A05	Security misconfiguration	Helmet defaults; non-prod features (Swagger, debug routes) gated by env
A06	Vulnerable components	Dependabot/ <code>npm audit</code> recommended in CI; Prisma & Nest pinned to current LTS
A07	Identification & auth failures	Refresh-token rotation w/ reuse detection; throttled <code>/auth/*</code>
A08	Software & data integrity	Lockfile committed; container image immutability via ACR digest pinning
A09	Logging & monitoring	Pino structured logs + App Insights traces; status_history audit
A10	SSRF	No server-side fetch of arbitrary URLs from user input

9. Frontend security

- React escapes by default; never use `dangerouslySetInnerHTML` with un-sanitised CMS HTML — sanitise via `DOMPurify` when needed.
- Tokens stored in memory + persisted refresh token in `localStorage`; acceptable trade-off for SPA. **Future:** switch refresh to `HttpOnly` secure cookie when the API is moved behind a same-origin reverse proxy.
- Stripe Elements collects card data — PAN never traverses our backend.
- CSP can be tightened on SWA via `staticwebapp.config.json` (recommended enhancement).

10. Operational hardening

- Container runs as non-root; minimal Node 20 alpine base image.
- Postgres firewall: only the Container App subnet + admin maintenance IP ranges; no public 0.0.0.0/0.
- Managed identity (`AcrPull`) used for image pulls — no ACR admin password in the cluster.

11. Compliance posture

- **UAE VAT** — 5% rate snapshot per order/invoice; invoice number monotonic; VAT amount on PDF.
- **Consumer rights / returns** — `ReturnsModule` enforces status transitions and timestamped audit fields.
- **GDPR-like requests** — soft-delete + admin reports cover a typical data-subject access request; full erasure is admin-driven.
- **PCI** — out of scope for the merchant: card data handled by Stripe Elements / Tabby / Tamara hosted flows.

12. Recommended next steps

- Add an automated dependency scan job (npm audit + Snyk).
- Move secrets to Azure Key Vault with rotation policies.
- Add CSP & SRI on SWA.
- Add WAF (Front Door / Application Gateway) in front of ACA.
- Quarterly restore drill on Postgres backups.